PukSquash Developer Manual

Coach Availability and Notice Board

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

Coaches publish the fixed time slots during which they can offer private coaching. Students browse those published times through the coach notice board. Availability describes a coach’s offer; it does not itself reserve a court.

An ACTIVE COACH submits a date and one or more time-slot IDs. The request validator rejects malformed or duplicate IDs and past dates. The controller checks active slots and times, skips availability already published, and creates missing rows. The coach can retrieve their published slots and remove an eligible slot. The notice board groups valid rows by coach and sorts slots for display.

1 1 Files and Responsibilities

resources/js/pages/CoachBookings.vue — availability management

resources/js/pages/CoachNoticeBoardPage.vue — coach notice board

app/Http/Requests/PublishCoachAvailabilityRequest.php — input rules

app/Http/Controllers/Api/CoachAvailabilityController.php — publish, mine, destroy and noticeBoard

app/Repositories/CoachAvailabilityRepository.php — named Data Connect operations

1 2 API Contract

PUT /api/coach/availability accepts availability_date and time_slot_ids. GET and DELETE /api/coach/availability manage the caller’s slots. GET /api/notice-board/coaches?date=YYYY-MM-DD lists coaches. GET /api/coaches/available also takes time_slot_id.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Validating published availability

app/Http/Requests/PublishCoachAvailabilityRequest.php | source line 26 onward; blank and comment lines omitted

```
public function rules(): array
    {
        return [
            'availability_date' => [
                'required',
                'date_format:Y-m-d',
                'after_or_equal:today',
            ],
            'time_slot_ids' => [
                'required',
                'array',
                'min:1',
            ],
            'time_slot_ids.*' => [
                'required',
                'string',
                'regex:/\A[0-9a-fA-F]{32}\z/',
                'distinct',
            ],
        ];
    }
}
```

Explanation

The FormRequest validates the date and selected slot identifiers before the controller writes anything. Authorisation remains in Firebase middleware and the controller's ACTIVE COACH check; authorize returning true does not grant every user coaching privileges.

Physical Code Explanation

1 availability_date must use YYYY-MM-DD and cannot be before today.

2 time_slot_ids must contain at least one item.

3 Each item must be a distinct 32-character hexadecimal string in this baseline. Do not substitute a hyphenated UUID in a test without matching the actual validator.

4 The controller separately checks that every requested ID belongs to an active TimeSlot and that today's selected times have not started.

Publishing returns created_time_slot_ids and existing_time_slot_ids. Repeating the same selection should not create duplicates. The controller batches missing rows through createMany and treats recognised duplicate errors as existing availability.

3 Composite availability identity

dataconnect/schema/schema.gql | source line 173 onward; blank and comment lines omitted

```
type CoachAvailability @table(key: ["coach", "timeSlot", "availabilityDate"]) {
  coach: User!
  timeSlot: TimeSlot!
  availabilityDate: Date!
  createdAt: Timestamp! @default(expr: "request.time")
}
```

Explanation

A CoachAvailability row is identified by coach, timeSlot and availabilityDate. That composite identity reflects the rule that one coach publishes a given date/slot once. Unlike ordinary Bookings, an availability row has no court relation.

Physical Code Explanation

1 The @table key declares the composite identity in the schema.

2 coach points to a User, so the server must enforce that the user is an ACTIVE COACH. A relation alone does not prove the role.

3 timeSlot points to the fixed slot shared with bookings. availabilityDate distinguishes the same hour on different days.

4 createdAt records when the availability was published.

The management response adds has_confirmed_booking. Removal checks for a confirmed coaching booking and returns 409 when one exists. The notice board filters out inactive coaches and inactive slots, deduplicates slots by ID and orders coaches by name. Published availability and actual bookability must still be distinguished at booking time.

4 Verification and Maintenance

Verification checklist

1 Publish valid slots as an ACTIVE COACH and verify both the management list and notice board. Repeat the request and check existing IDs are reported without duplicates.

2 Submit as USER or ADMIN: the active-coach endpoints must return 403.

3 Try duplicate, malformed, inactive and past slot selections. Verify rejection before new availability is created.

4 Remove an unbooked slot, then try removing one with a confirmed private session: expect success and 409 respectively.

5 Run PublishCoachAvailabilityTest, PublishCoachAvailabilityRequestTest and ManageCoachAvailabilityTest; separately verify the real Data Connect behaviour because mocked repositories do not test the deployed connector.

Implementation boundaries

Today comparisons in publishing use the Laravel application timezone, while several booking paths explicitly use Africa/Johannesburg. Configure and test the timezone consistently.

Batch publication uses separate operations and is not an all-or-nothing transaction. A non-duplicate failure can leave earlier successful rows published; handle retry and feedback accordingly.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).